K U S I N A K O D E

Kusina Kode System Primer

A lecture-style study module for the capstone defense. Every concept is taught the same way, and every example points to code that actually runs in Kusina Kode: the Android app, the PHP API on Azure, and the admin console.

1
Definition
The textbook concept.

2
Why it matters here
What problem it solves in Kusina Kode.

3
How we applied it
The real file, class or table.

4
In plain words
How to say it to a non-programmer.

5
Insights
Why we chose it, and supporting evidence.

BEFORE YOU START

Study plan

Six sessions of about 45 minutes. Read a unit, then answer its "Panel might ask" lines out loud without looking. Units 1 to 3 are the program itself; units 4 and 5 are how it is protected and hosted, and unit 6 ties it all together in the diagrams.

SESSION	UNIT	GOAL	YOU CAN EXPLAIN…
1	OOP & architecture	Know how the code is organised	Why the app is split into screens, ViewModels, rules and repositories
2	Data structures	Know where data lives and why	Why a map, a set or a chain was the right container
3	Algorithms	Know the core logic step by step	How a guess becomes coloured tiles, and how XP and ranks are computed
4	Cybersecurity	Know what protects players and rewards	Passwords, tokens, lockout, encryption, and why the ledger can't be faked
5	Cloud computing	Know where it runs and how well	Each Azure service, what it costs us, and what the stress test showed
6	System diagrams	Walk through each figure in a minute	What every shape means in each diagram type, and how to walk through ours

UNIT 1

Object-oriented programming & architecture

The app is written in Kotlin and the admin console in C#; both are object-oriented. In the app alone there are 114 data classes, 15 interfaces, 16 enums, 59 singleton objects and 8 ViewModels.

1.1 Classes and objects `data class · enum · object`

DEFINITION

A **class** is a blueprint that bundles data (properties) with the behaviour that works on it (methods). An **object** is one instance made from that blueprint.

WHY IT MATTERS HERE

Kusina Kode handles many "things" with their own data: levels, players, badges, ingredients, attempts. Modelling each as a class keeps its data and rules together and consistent everywhere they are used.

HOW WE APPLIED IT

- **Data classes** for records, e.g. `PlayerProgress` (points, streaks, solved levels) and `LevelData`. Kotlin generates equality and copying for them, so `current.copy(totalPoints = …)` makes a new updated record safely.
- **Enums** for fixed choices: `Region` (Luzon, Visayas, Mindanao), `Rarity`, `TileState` (Correct, SemiCorrect, Wrong), `PowerUp`.
- **Singleton objects** for one shared rulebook: `WordleEngine`, `ScoreRules`, `ChefRank`, `IngredientCatalog`.

IN PLAIN WORDS

A class is the recipe card; an object is the dish you cook from it. Every level in the game is cooked from the same "level" recipe, so they all have a word, a region and ingredients.

INSIGHTS

Data classes are **immutable by habit** in the app: progress is replaced with `copy()` rather than edited in place, which avoids half-updated scores when several screens read the same data.

Enums stop impossible values: a tile can only be one of three states, so a typo like "corect" can't compile.

Panel might ask: "Why use a singleton for the scoring rules?" Because there must be exactly one rulebook; two copies could score the same round differently.

1.2 Encapsulation `private · read-only views`

DEFINITION

Hiding an object's internal state and exposing only controlled ways to read or change it.

WHY IT MATTERS HERE

Game state (the board, the timer, the KK balance) must only change through the game's rules; a screen should never be able to set "you won" directly.

HOW WE APPLIED IT

```
// GameViewModel.kt
private val _uiState = MutableStateFlow(…)   // only the ViewModel can change it
val uiState = _uiState.asStateFlow()         // screens get a read-only view
```

- The same pattern guards `PantryViewModel`, `KkBalance` and the gamification store (`private set` on `gamification` in `KusinaKodeApp`).
- On the web, ASP.NET pages reach data only through the composition root and service classes (`DishService`, `IngredientService`), never straight to SQL.

IN PLAIN WORDS

The kitchen has one door for orders. Diners can see the food, but only the cook can change it.

INSIGHTS

This is also a security property: if the screen can't change the score, a bug in a screen can't award free KK. The server re-checks everything anyway (see 4.2).

1.3 Abstraction `interfaces`

DEFINITION

Describing *what* something does through an interface while hiding *how* it does it.

WHY IT MATTERS HERE

The game needs data from several places: the phone's storage, the PHP API, the chain. The game logic shouldn't care which one answered.

HOW WE APPLIED IT

- 15 repository interfaces, e.g. `GamificationRepository`, `PantryRepository`, `LevelRepository`, `AuthRepository`, `WalletRepository`.
- The coordinator only knows the interface: `GamificationCoordinator(repository: GamificationRepository, …)`.
- The web console does the same with `IIngredientStore`, `IReelStore` and `IAdminSignIn` in `Web_KusinaKode/Abstractions`.

IN PLAIN WORDS

> The game asks "give me the pantry". It doesn't need to know whether the answer came from the phone or from the server.

INSIGHTS

Abstraction is what made the unit tests possible: tests swap in fake repositories, so `ProgressRules` and `BadgeRules` are tested without a phone or internet.

It also let the team move from local XAMPP to Azure without rewriting the game logic.

1.4 Inheritance `extends Android & ASP.NET base classes`

DEFINITION

A class reusing and extending another class's behaviour (a subclass "is a" kind of its parent).

WHY IT MATTERS HERE

Android and ASP.NET already provide lifecycle, memory and request handling. Inheriting from their base classes gives Kusina Kode those features for free.

HOW WE APPLIED IT

- `class KusinaKodeApp : Application()` : runs once when the app starts (sets up the network status, gamification store, chain coordinator).
- `class GameViewModel(…) : ViewModel()` and seven other ViewModels: survive screen rotation and are cleared automatically.
- Each admin page (`Levels.aspx.cs` , `Audit.aspx.cs` …) inherits from ASP.NET's `Page` ; the shared `Admin.master` layout gives every page the sidebar and sign-in check.

IN PLAIN WORDS

Instead of building our own kitchen, we rented one and added our own recipes.

INSIGHTS

The code favours **composition over deep inheritance**: our own classes rarely extend each other. They combine small parts (a repository, a coordinator) instead, which keeps them easier to test and change.

1.5 Polymorphism `one interface, many forms`

DEFINITION

One interface or type can stand for many concrete forms, and the right behaviour is chosen at run time.

WHY IT MATTERS HERE

Some features have several sources or outcomes: the same "progress" can be stored locally or remotely, and a guess can be valid or invalid in several ways.

HOW WE APPLIED IT

- **Interface polymorphism:** `LocalGamificationRepository`, `RemotePantryRepository`, `RemoteAttemptRepository` are concrete forms of their repository interfaces.
- **Sealed types:** `WordleEngine.GuessValidation` is `Valid`, `Incomplete`, `WrongLength(expected, actual)` or `NotLetters`. The compiler forces every screen to handle all four.
- `NotificationGlyph` is either `Art` (a bundled picture) or `Remote` (a web picture), drawn differently.

IN PLAIN WORDS

"Serve the dish" works for adobo or sinigang; each one knows how it should be plated.

INSIGHTS

Sealed types turn "did we handle every case?" into a compile error instead of a crash. When a new validation case is added, every place that forgot it stops compiling.

1.6 MVVM, Repository & Clean Architecture `design patterns`

DEFINITION

MVVM separates the View (screen), ViewModel (state and actions) and Model (data and rules). The **Repository** pattern hides data sources behind one class. **Clean Architecture** arranges code in layers where inner rules never depend on outer details. The **Observer** pattern lets screens react automatically when data changes.

WHY IT MATTERS HERE

The project rule (`CLAUDE.md`) is that screens never call the network or database directly. Four people work on different modules, so clear layers prevent stepping on each other.

HOW WE APPLIED IT

- **App:** Compose screens → ViewModels exposing `StateFlow` (observer) → repositories → `KusinaApi` (Ktor HTTP client). Pure rules live in `domain/` with no Android code.
- **Web console:** Kyla split it into `KusinaKode.Domain` (rules, with tests), `Application` (services), `Data` (stores) and `Presentation` (pages), wired in one composition root.

IN PLAIN WORDS

> The waiter (screen) takes the order, the head cook (ViewModel) decides what happens, and the pantry staff (repository) fetch ingredients. Nobody does someone else's job.

INSIGHTS

Because the rules are separate, the same tile-scoring and XP logic is unit-tested on its own (`WordleEngineTest` , `ScoreRulesTest` , `ProgressRulesTest`). Defect D-01 (wrong colours for repeated letters) was found and fixed in that layer.

Panel might ask: "Why not let screens call the API?" A slow network would freeze the screen, and every screen would duplicate error handling. The ViewModel layer does both in one place.

UNIT 2

Data structures

Choosing the right container decides how fast the app can find, check and store things. Each structure below is used on purpose.

2.1 Lists and arrays `the guess grid`

DEFINITION

An ordered collection where each element has a position (index 0, 1, 2…).

WHY IT MATTERS HERE

A guess is a row of letters, and each letter's colour depends on its **position**. Order is the whole point.

HOW WE APPLIED IT

`WordleEngine.evaluate()` returns `List<TileState>`, one state per letter, built with `MutableList(ans.length) { TileState.Wrong }`. The board is a list of these rows: six rows for every dish (`MAX_ATTEMPTS = 6`).

IN PLAIN WORDS

A row of numbered boxes. Box 1 checks letter 1, box 2 checks letter 2, and so on.

INSIGHTS

The list is sized from the answer's length, not fixed. That is how the project meets its **variable word length** requirement: ADOBO makes 5 boxes, SINIGANG makes 8, with no special cases.

2.2 Hash maps `key → value in one step`

DEFINITION

A map stores key-value pairs and finds a value from its key in roughly constant time, O(1), no matter how many entries it holds.

WHY IT MATTERS HERE

The app constantly looks things up by name or id: a letter's remaining count, an ingredient's picture, an ingredient's details.

HOW WE APPLIED IT

- `remaining = mutableMapOf<Char, Int>()` in the tile algorithm counts unused answer letters.
- `IngredientArt.byName` maps about 200 dataset ingredient names to their photos.
- `IngredientCatalog.byId` maps ids like `ing_tuna_fresh` to ingredient records.

IN PLAIN WORDS

A labelled spice rack: you go straight to "garlic" instead of opening every jar.

INSIGHTS

Scanning a list of 123 ingredients for every tile on a dish page would repeat that work for each tile; a map answers each lookup at once. Maps also made the 1.23 fix simple: when a web name isn't in the picture map, the app looks it up by id instead.

2.3 Sets `membership without duplicates`

DEFINITION

A collection of unique items, built for fast "is X in here?" checks.

WHY IT MATTERS HERE

"Has this player already solved this dish?" decides whether a win pays XP and KK. It must be fast and must never count a dish twice.

HOW WE APPLIED IT

`PlayerProgress.solvedLevels` is a `Set<Int>`. In `ProgressRules.advance()`: `if (event.levelId in current.solvedLevels) return current`, so a replay changes nothing. `regionsSolved` and `newIngredientIds` are sets too.

IN PLAIN WORDS

A stamp card: each dish gets one stamp, and stamping it again doesn't add a second.

INSIGHTS

This set is the anti-farming rule. Defect D-05 ("replaying a solved dish awarded points") was fixed by checking this set first. The stress test confirmed it: replays showed "Already solved, +0".

2.4 Streams and queues `StateFlow · event bus · chain queue`

DEFINITION

A **queue** processes items in arrival order (first in, first out). A **stream** (flow) delivers a sequence of values over time to whoever is listening.

WHY IT MATTERS HERE

Many things happen after a win: XP, badges, KK, the chain receipt, the palayok. They must happen in order, even if the player is already on another screen.

HOW WE APPLIED IT

- Screens observe `StateFlow` streams; when the value changes, the screen redraws.
- `GameEvents` / `GamificationEvents` broadcast "round completed", and `GamificationCoordinator` listens for the whole app lifetime.
- `ChainQueue` lines up reward transactions for the ledger; on the server, a job settles anything still pending every 15 minutes.

IN PLAIN WORDS

Orders go on a rail in the kitchen and are cooked one after another, so none are lost when it gets busy.

INSIGHTS

Defect D-31 ("two rewards at once both fail") is exactly the problem queues solve: two writes racing for the same ledger. Ordering them, plus the server-side settle job, removes the race.

2.5 Hash-linked chain `blocks.jsonl`

DEFINITION

A linked list where each element (block) stores the **hash of the previous block**, so the whole list is chained together. This is the core data structure of a blockchain.

WHY IT MATTERS HERE

The study's third objective is a tamper-resistant reward record. Every KK earned or spent must be provable later.

HOW WE APPLIED IT

The server keeps the chain in `/var/lib/kusinakode/chain/blocks.jsonl`, one block per line, with each transaction in `txs/`. `lib/chain.php` computes each block's hash and checks the chain's tip; the database stores only receipts (`tx_ref` in `reward_claims`, `shop_unlocks`, `ingredient_draws`).

IN PLAIN WORDS

A receipt book where each page carries a fingerprint of the page before it. Tear out or change one page and every page after it stops matching.

INSIGHTS

Why a file and not a SQL table? Because the database is meant to be the editable working copy, while the chain is the evidence. An admin can edit a table, but cannot silently edit a block without breaking every later hash. The hourly integrity check on the Audit page detects exactly that.

2.6 Relational tables `20 tables · keys · relationships`

DEFINITION

Data stored in tables of rows and columns, linked by **primary keys** (unique id per row) and **foreign keys** (a column pointing at another table's key).

WHY IT MATTERS HERE

Players, levels, attempts, rewards and ingredients all relate to each other, and the admin console, analytics and leaderboard query across them.

HOW WE APPLIED IT

Azure SQL holds 20 tables (see the ERD). Examples: `attempts.user_id → users.id`, `attempts.level_id → levels.id`, enforced with `ON DELETE CASCADE`. Composite keys stop duplicates, e.g. `reward_claims`' key is `(user_id, kind, claim_key)`, so one first-dish bonus per player per dish.

IN PLAIN WORDS

Linked spreadsheets: the attempts sheet doesn't copy player names, it just points to the player's row.

INSIGHTS

The composite key on `reward_claims` makes double payouts impossible at the database level, not just in code. Cascading deletes explain a real event: deleting a level also deleted its attempts, which is why the console now refuses to remove played levels.

UNIT 3

Algorithms

The step-by-step procedures behind the game. Each one is small, deterministic and unit-tested.

3.1 Two-pass tile scoring `WordleEngine.evaluate · O(n)`

DEFINITION

A **multi-pass algorithm** walks the data more than once, each pass doing one job, so later passes can rely on what earlier passes decided.

WHY IT MATTERS HERE

This is the heart of the puzzle: turning a guess into green (correct), yellow (in the word, wrong place) and grey (not in the word), and it must handle repeated letters correctly.

HOW WE APPLIED IT

```
Pass 1: for each position i
          if guess[i] == answer[i] → Correct (green)
          else count answer[i] in remaining
Pass 2: for each position not yet green
          if remaining[guess[i]] > 0 → SemiCorrect (yellow),
remaining -1
          else → Wrong (grey)
```

Example: answer **SISIG**, guess **SINGS** → S I green, N grey, G yellow, the second S yellow (exactly what the booth screenshot shows).

IN PLAIN WORDS

First mark every letter that's in the exact right spot. Then, with the letters left over, mark the ones that exist somewhere else, and never more times than they appear in the answer.

INSIGHTS

The original Capstone 1 code used a one-pass `indexOf()`, which matched the first occurrence and showed too many yellows for repeated letters (defect D-01). Two passes plus a count map fix it.

Time is **O(n)** for an n-letter word: two passes, each with constant-time map lookups. That's instant even for long dish names.

Panel might ask: "Why two passes?" Because greens must be claimed first; otherwise a yellow could use up a letter that a later position needs for a green.

3.2 The XP formula `ScoreRules.pointsFor`

DEFINITION

A **deterministic scoring function**: the same inputs always give the same output, with no randomness.

WHY IT MATTERS HERE

XP must reward skill fairly and be reproducible: the same round always scores the same on any phone, which the acceptance criteria call "consistent across sessions".

HOW WE APPLIED IT

```
XP = 100                          // solved the dish
   + 20 × (6 - tries used)        // tries left over
   + 50 if solved on the 1st try
   + 60 (<30 s) / 40 (<60 s) / 20 (<120 s) / 0
```

Range: **100** (last try, slow) to **310** (first try, under 30 s). Losses and replays score 0.

IN PLAIN WORDS

You get 100 for solving it, plus bonuses for needing fewer tries, nailing it first time, and being quick.

INSIGHTS

Bonuses use **buckets** (under 30 s, under 60 s…) rather than exact seconds, which the code comments say makes a score "easy to explain to a player, and easy to re-derive if the chain ever needs to audit it".

3.3 Multi-key sorting `LeaderboardRules.rank`

DEFINITION

Sorting by a primary key and breaking ties with secondary keys, using a stable comparison sort (O(n log n)).

WHY IT MATTERS HERE

A leaderboard must give everyone a clear, fair position, even when two players have the same points.

HOW WE APPLIED IT

```
rows.sortedWith(
  compareByDescending { it.points }      // 1. most XP
    .thenByDescending { it.correctCount } // 2. more dishes
solved
    .thenBy { it.name.lowercase() })      // 3. A-Z, so ties
are stable
```

IN PLAIN WORDS

Rank by score; if tied, whoever solved more dishes goes higher; if still tied, alphabetical.

INSIGHTS

The final alphabetical key means the order never "flickers" between refreshes for tied players, which players would otherwise read as a bug.

3.4 Threshold lookup & gated progression

`ChefRank · UnlockManager`

DEFINITION

Mapping a number onto ordered ranges (thresholds), and a **state machine** that only allows the next step when the current one is finished.

WHY IT MATTERS HERE

Ranks and level unlocking are the progression that motivates players (objective 2).

HOW WE APPLIED IT

- `ChefRank`: Kusinero 0 → Line Cook 5 → Sous Chef 10 → Head Chef 20 dishes → Kusina Master when every visible dish is solved. The steps match the round badges, so a rank and its badge arrive together.
- `UnlockManager.unlockNextLevel()` unlocks only level N+1, and only if the level just won is the current highest N.

IN PLAIN WORDS

Like a cooking school: you move from apprentice to head chef as you master more dishes, and each class opens only after you pass the one before.

INSIGHTS

Ranks count **dishes solved, not XP**, so a rank reflects how much of the cuisine you've learned, not how fast you type. "Kusina Master" is tied to the total of visible dishes, so it adjusts automatically when admins add levels.

3.5 String normalisation & matching `IngredientArt.forName`

DEFINITION

Converting text to one canonical form (lowercase, letters and digits only) before comparing, often with regular expressions, plus a **fallback chain** when the first lookup fails.

WHY IT MATTERS HERE

Ingredient names arrive written in different ways: "White vinegar", "WHITE VINEGAR", "Vinegar (White/Cane/Coconut)" from the web console.

HOW WE APPLIED IT

```
key = name.lowercase().replace(Regex("[^a-z0-9]+"), "
").trim()
1. look up key in the picture map
2. else find the ingredient with that name in the catalogue →
its id
3. else try "ing_" + key as an id
→ pantry picture for that id, or null (drawn icon)
```

IN PLAIN WORDS

Ignore capitals and punctuation, then try three ways to recognise the ingredient before giving up and drawing a simple icon.

INSIGHTS

Before 1.23, 55 of the web's 123 ingredient names showed a plain icon because only step 1 existed. The test `every ingredient in the book has a picture by its name` now guards all 123.

3.6 Hashing & digital signatures SHA-256 · ECDSA

DEFINITION

A **cryptographic hash** (SHA-256) turns any data into a fixed 64-character fingerprint; changing one character changes the whole fingerprint. A **digital signature** (ECDSA) proves who created a message and that it wasn't altered.

WHY IT MATTERS HERE

They are the algorithms that make the reward ledger trustworthy.

HOW WE APPLIED IT

- Each receipt's id is `'0x' . hash('sha256', kk_canonical_json($receipt))`: canonical JSON first, so the same receipt always hashes the same.
- Blocks are signed with `openssl_sign(…, OPENSSL_ALGO_SHA256)` using an elliptic-curve key on **secp256k1** (the curve Ethereum and Polygon use), with prime256v1 as a fallback on servers without it.
- Custodial wallet addresses are derived from the public key's SHA-256 hash.

IN PLAIN WORDS

The hash is a fingerprint of the receipt; the signature is the system's stamp. Change the receipt and the fingerprint won't match; fake the stamp and it won't verify.

INSIGHTS

Canonical JSON matters: the same data with keys in a different order would otherwise produce a different hash and look "tampered". The unit suite (`api/tests/run.php`) checked these primitives: 51 passed, 0 failed.

UNIT 4

Cybersecurity

Security follows the CIA triad: **confidentiality** (only the right people see data), **integrity** (data isn't changed without detection) and **availability** (the system stays usable). Each module below protects one or more of the three.

4.1 Authentication `bcrypt · bearer tokens · Google · Entra`

DEFINITION

Proving a user is who they claim to be, and keeping that proof safe afterwards.

WHY IT MATTERS HERE

Accounts hold KK, badges and progress. A stolen account is stolen rewards.

HOW WE APPLIED IT

- Passwords are stored only as **bcrypt hashes** (`password_hash(…, PASSWORD_DEFAULT)`) and checked with `password_verify`; the plain password is never stored.
- After login the server issues a random token (`random_bytes`), stores only its **hash** in `api_tokens.token_hash`, and expires it after 30 days. Logging out deletes it.
- The phone sends it as `Authorization: Bearer …` (`KtorClient`).
- Players can sign in with Google (verified server-side); admins sign in to the console with Microsoft Entra.

IN PLAIN WORDS

Your password is turned into an unreadable fingerprint before it's saved. After you log in, you get a temporary pass, and the server only keeps a fingerprint of that too.

INSIGHTS

Hashing tokens server-side means even a leaked database can't be used to log in as players. Before the 17 Aug security review, signing out left tokens valid for 30 days (D-25); now logout revokes them.

4.2 Authorization `identity from the token, never the request`

DEFINITION

Deciding what an authenticated user is allowed to do. The server must derive "who you are" from a trusted credential, not from what the client says.

WHY IT MATTERS HERE

Every reward is tied to a user id. If a client could send any `user_id`, it could write progress into someone else's account.

HOW WE APPLIED IT

- Endpoints call `kk_require_user()`, which reads the bearer token and ignores any `user_id` in the body (fix for D-14, then D-17 for the last four endpoints).
- Admin console: every page re-checks the signed-in admin and their allowed sections (`AdminAuth.IsActiveAdmin`), fails closed if unsure, and signs out after 15 minutes idle.
- Players never touch wallet keys: wallets are **custodial**, managed on the server.

IN PLAIN WORDS

The server doesn't believe "I'm player 46". It checks your pass and decides for itself who you are.

INSIGHTS

D-14 was rated **Critical**: "identity was a claim rather than a credential". It's a classic example of broken access control, the top risk on the OWASP Top 10.

4.3 Brute-force & enumeration defence

`lockout · rate limits · generic errors`

DEFINITION

Limiting repeated guesses and avoiding messages that reveal whether an account exists.

WHY IT MATTERS HERE

Login and password-reset codes can be guessed by a script if attempts are unlimited.

HOW WE APPLIED IT

- Every attempt is logged in `auth_attempts`; too many failures in a time window locks that account for a set number of minutes ("Too many attempts. Try again in … minutes").
- Lockout is per account, not per network (D-33): one person's mistakes on campus Wi-Fi don't lock everyone else.
- Reset codes are bcrypt-hashed and never returned in the API (D-20). Sign-up no longer says which emails exist (D-22), and database errors aren't shown to clients (D-23).

IN PLAIN WORDS

Get the password wrong too many times and that account takes a short break. And the app never confirms whether an email is registered.

INSIGHTS

Returning the reset code in the API response (D-20) meant anyone could reset anyone's password: a full account takeover. It was found by a security review, not by a user, which is why reviews are part of testing.

4.4 Data protection `TLS · AES-256 · secrets · privacy`

DEFINITION

Keeping data confidential **in transit** (encryption over the network) and **at rest** (encryption on disk), and keeping secrets out of code.

WHY IT MATTERS HERE

Tokens, emails and wallet keys pass through phones, Wi-Fi networks and a public GitHub repository.

HOW WE APPLIED IT

- **In transit:** HTTPS with Let's Encrypt certificates (certbot on the API VM); the app's `network_security_config` governs its connections.
- **At rest on the phone:** the token lives in `EncryptedSharedPreferences` (AES-256-GCM, key in the Android Keystore), and `allowBackup="false"` keeps it out of device backups (D-24).
- **At rest on the server:** reward payloads use AES-256-GCM; the master key and wallet private keys live in a vault (`lib/vault.php`), "never in source, in SQL".
- **Repo hygiene:** keystore and passwords are git-ignored; Azure resource paths were masked before publishing test data.
- **Privacy:** UAT responses are confidential under the Data Privacy Act of 2012 (RA 10173).

IN PLAIN WORDS

Everything travels in a locked envelope, the phone keeps your pass in a safe, and the server's master keys are never written into the code.

INSIGHTS

GCM mode adds authentication on top of encryption: the test suite checks that "tampering ciphertext fails AES-256-GCM authentication", so altered data is rejected rather than silently decrypted into garbage.

4.5 Injection prevention `prepared statements`

DEFINITION

SQL injection is when user input is treated as part of a database command. **Prepared statements** send the command and the data separately, so input can never become code.

WHY IT MATTERS HERE

Usernames, guesses, report messages and admin form fields all reach the database.

HOW WE APPLIED IT

64 PHP files on the live API use PDO `prepare()` with named parameters, e.g. `$pdo->prepare("DELETE FROM api_tokens WHERE token_hash = :h")`. The guess validator also accepts letters only (`NotLetters`).

IN PLAIN WORDS

The database gets the question and the answer in separate envelopes, so someone typing a command into the name box just gets a strange name, not a deleted table.

INSIGHTS

Injection is a long-standing entry on the OWASP Top 10. Validating on the phone is for user experience; the server's prepared statements are the real defence, because attackers can skip the app entirely.

4.6 Ledger integrity & auditing `hash chain · PoA · hourly check`

DEFINITION

Integrity means data can't be changed without detection. **Proof of Authority (PoA)** is a blockchain model where approved signers (here, the server) create blocks, instead of public mining.

WHY IT MATTERS HERE

KK and badges should not be editable by anyone, admins included. This is the study's core contribution.

HOW WE APPLIED IT

- Each reward is encrypted, hashed and signed (`kk_phase2_assemble_sign_poa`), then appended to the hash-linked chain under a lock so two writes can't collide.
- The admin Audit page lists the whole ledger, and an hourly whole-chain check shows a **Chain integrity** card.
- The phone only ever shows balances and status read from the API: no seed phrases, keys or gas fees (a project non-negotiable).

IN PLAIN WORDS

Every reward is written in ink in a numbered, fingerprinted book. If anyone erases a line, the hourly check spots it.

INSIGHTS

Why PoA instead of a public chain like Ethereum? No gas fees per reward, instant confirmation, and no player wallets to lose, while still keeping the tamper-evidence that the study needs. secp256k1 keys keep the door open to a Polygon-compatible chain later.

Panel might ask: "Can an admin give themselves KK?" They could edit the database, but not the chain; the receipt would be missing or the hashes would break, and the integrity check would flag it.

UNIT 5

Cloud computing

Kusina Kode runs on Microsoft Azure in the `kusina-kode` resource group (Southeast Asia). This unit maps each service to the concept it demonstrates and to the stress-test evidence.

5.1 Cloud service models `IaaS · PaaS · SaaS`

DEFINITION

IaaS rents raw servers you manage yourself. **PaaS** rents a managed platform where the provider runs the server and you supply the app or data. **SaaS** is finished software you just use.

WHY IT MATTERS HERE

Each part of the system has different needs, so it uses a different model.

HOW WE APPLIED IT

PART	AZURE SERVICE	MODEL
PHP REST API + reward chain	Virtual Machine `vm-kusinakode-api`	IaaS
Database (36 tables)	Azure SQL `sql-kusinakode/kusinakode`, Basic tier	PaaS
Admin console	App Service `app-kusinakode-admin2`, B1	PaaS
Uploaded pictures	Blob Storage `stkusinakode/media`	PaaS (storage)
Admin sign-in	Microsoft Entra ID	SaaS identity

IN PLAIN WORDS

We rent a whole kitchen for the API (and run it ourselves), but rent ready-made services for the database, website and photo storage.

INSIGHTS

The API is on a VM because the chain needs a private folder on disk and scheduled jobs (cron), which a full server allows. The database and console are PaaS so Azure handles patches and backups.

5.2 Compute: the API virtual machine `vm-kusinakode-api`

DEFINITION

A virtual machine is a software-defined computer in the provider's data centre, with its own OS, disk and network address.

WHY IT MATTERS HERE

It runs everything the phone talks to: `api.kusinakode.com/kusinakode/REST/`.

HOW WE APPLIED IT

- A Linux VM serving the PHP API from `/var/www/kusinakode`, with the chain in `/var/lib/kusinakode/chain`.
- **Scheduled jobs:** `settle_pending_cron.sh` every 15 minutes (finishes stuck rewards) and a receipt-cache refresh every minute.
- **Operations:** changes are applied with backups first (`*.bak-YYYYMMDD`) through Azure's Run Command.

IN PLAIN WORDS

A computer in Microsoft's building that stays on 24/7 and answers the app's requests.

INSIGHTS

Stress test: at 40 simultaneous players the VM's CPU stayed at or under about 12%. The server is not the bottleneck.

5.3 Managed relational database `Azure SQL · DTU`

DEFINITION

A database service where the provider runs the database engine. Capacity is sold in tiers; Azure's **DTU** (Database Transaction Unit) measures a blend of CPU, memory and disk work.

WHY IT MATTERS HERE

Every level list, leaderboard and reward lookup hits this database.

HOW WE APPLIED IT

Azure SQL Database `kusinakode` on the **Basic** tier, reached by the API's PHP code and Kyla's console. Firewall rules allow only known IPs; administrators sign in with Entra ID or the SQL admin login.

IN PLAIN WORDS

A filing cabinet Microsoft looks after, where every record of the game lives.

INSIGHTS

Stress test: database load rose from 7% (5 players) to **91% peak at 40 players**, while the VM sat near 10%. The database tier is the system's capacity limit. The fix is **vertical scaling** (Basic → Standard), a settings change that costs more per month.

Panel might ask: "How many users can it handle?" Up to 40 at once with zero errors (average response 1.6 s). Past about 50, the Basic database would start queuing; raising its tier is a configuration change.

5.4 Object storage `Blob Storage`

DEFINITION

Storage for files ("blobs") addressed by URL, cheaper and simpler than keeping files on a server's disk.

WHY IT MATTERS HERE

Admins upload dish, equipment and ingredient pictures that the console and app must show.

HOW WE APPLIED IT

Storage account `stkusinakode`, container `media`, folders `levels/` and `equipment/`. Files are readable by direct URL, but the container can't be listed anonymously.

IN PLAIN WORDS

An online photo album: each picture has its own link, but strangers can't browse the album.

INSIGHTS

Keeping pictures out of the database keeps the database small and fast, which matters given that the database is the bottleneck (5.3).

5.5 Identity, domains & certificates `Entra · DNS · TLS`

DEFINITION

Identity as a service handles sign-in centrally. **DNS** maps names like `api.kusinakode.com` to servers, and **TLS certificates** prove a site is genuine and encrypt traffic.

WHY IT MATTERS HERE

Players and admins need memorable, trusted addresses, and admin sign-in needs strong protection.

HOW WE APPLIED IT

- `api.kusinakode.com` → API VM. The app first used a hard-coded campus network IP (D-19), then the Azure VM's IP, and from 1.16 this name.
- `admin.kusinakode.com` → App Service, with Microsoft Entra sign-in for admins.
- Certificates from Let's Encrypt on the VM, renewed automatically by certbot.

IN PLAIN WORDS

The system has proper addresses with a padlock, and admins log in with their Microsoft account.

INSIGHTS

Using a domain name instead of an IP means the server can move or be replaced without releasing a new app; only the DNS record changes.

5.6 Monitoring, performance & scaling

`Azure Monitor · stress test`

DEFINITION

Monitoring collects metrics (CPU, memory, response time). **Stress testing** pushes a system past normal load to find its limits. **Scaling** adds capacity: vertically (a bigger machine or tier) or horizontally (more machines).

WHY IT MATTERS HERE

The study must show the system works for real users, not just on one developer's phone.

HOW WE APPLIED IT

TEST	RESULT
API, 5 → 40 players at once	0 errors · 0.25 s → 1.6 s average
Database load (Azure Monitor)	peak 91% DTU at 40 players
API VM CPU	≤ ~12%
Phone (vivo V17 Pro), 22 min of play	0 crashes · memory flat 146–153 MB
Cold start	0.7 s phone · 2.6 s emulator
Idle server, first request	5.4 s, then ~0.23 s

IN PLAIN WORDS

We pretended 40 people played at once and watched the gauges. Nothing broke; the database was the part working hardest.

INSIGHTS

Measuring, not guessing, found the real limit (the database, not the server) and two real fixes: the 1.26 startup change and the 1.27 image and animation changes. It also showed the Game Map is graphics-bound on a mid-range phone. The full workbook is `docs/testing/stress-mobile-2026-10-05/`.

Sources: the Kusina Kode app repository (Kotlin, release 1.27), the KusinaKode-Web repository (ASP.NET + PHP API), the live API on `vm-kusinakode-api` (read-only inspection, 8 Oct 2026), `docs/testing/defects.csv`, and the 5 Oct 2026 stress test workbook.

UNIT 6

System diagrams

A refresher on each kind of diagram in the paper (what it is for, what every shape means, and how to read it), followed by our own diagram and a walk-through. Everything was checked against the code and the live system on 15 Oct 2026.

6.0 How to read any diagram `the 4 questions`

WHY DIAGRAMS

A diagram is a **model**: it shows one view of the system and leaves the rest out on purpose. Each type answers a different question, so the panel will ask about each one differently.

PICK THE RIGHT ONE

QUESTION	DIAGRAM	FAMILY
Where does the **data** go, and where is it kept?	Data flow diagram (DFD)	Structured analysis
How is the **database** organised?	Entity relationship diagram (ERD)	Data modelling
Who does **which step** in one scenario, in order?	Process flow (swimlane flowchart)	Flowchart (ANSI/ISO 5807)
How does data move through **stages and jobs**?	Data pipeline	Data engineering
Who uses the system, and for **what**?	Use case diagram	UML (behaviour)
What are the **parts**, and how do they connect?	System architecture	Architecture
What are **all the paths** one user can take?	Activity diagram	UML (behaviour)
Where does it **run**, and how is it **protected**?	Cloud and cybersecurity architecture	Architecture
What went **in**, what did we **do**, what came **out**?	Conceptual framework (IPO)	Research

THE 4 QUESTIONS

- **What is the scope?** Read the title and the boundary first: the whole system, one process, or one scenario.

- **What are the shapes?** Every notation has a small set of symbols. Learn them below; a shape always means the same thing within one diagram type.
- **Where do I start?** At the start node, the actor, or the outside entity. Then follow the arrows.
- **What does a line mean?** It is different per type: data moving (DFD), a relationship (ERD), the next step (flowcharts), "uses" (use case), or a network call (architecture).

IN PLAIN WORDS

Each diagram is a different photo of the same house: the floor plan, the plumbing, the wiring, the guest list. None of them is the whole house, and each one is drawn with its own symbols.

PANEL MIGHT ASK

Why so many diagrams? Each answers a different question. The ERD cannot show the order of steps, and the activity diagram cannot show the tables.

Process flow vs activity diagram? A process flow follows one scenario across the people and systems that handle it (swimlanes). An activity diagram (UML) shows every path one user can take, including parallel steps (fork and join).

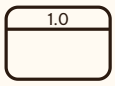
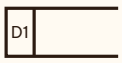
6.1 Data flow diagram (DFD) `Gane & Sarson · Levels 0, 1, 2`

WHAT IT IS

A DFD shows how **data moves** through a system: where it comes from, which process changes it, and where it is stored. It does **not** show order, timing or decisions; that is what the flowcharts are for.

It is drawn in **levels**. **Level 0** (context diagram) is the whole system as one process. **Level 1** opens that box into the main processes. **Level 2** opens one Level 1 process further (ours explodes 4.0 Process Rewards into 4.1 to 4.5).

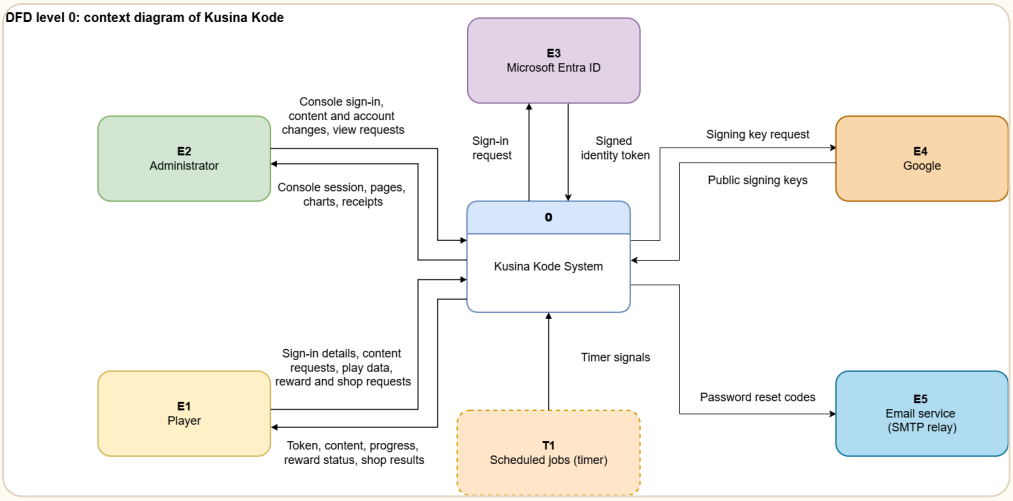
SYMBOLS

SYMBOL	NAME	WHAT IT MEANS
	**External entity**	A person or outside system that sends or receives data but is not part of the system (E1 Player, E3 Microsoft Entra ID). Also called a source or sink.
	**Process**	Something that **changes** data. Numbered by level: 1.0 at Level 1, 4.1 at Level 2. Named with a verb: "Authenticate & Authorize".
	**Data store**	Data at rest: a table group, a file or a folder (D1 Accounts, D5 Reward Ledger). Named with a noun.
	**Data flow**	Data in motion, labelled with a noun: "Credentials", "Reward status & TxHash". The arrow points where the data goes.
	**Two-way flow**	Data goes both ways (read and write). Clearer when drawn as two one-way arrows, each with its own label.
	**Timer (our convention)**	T1 Scheduled Jobs, drawn dashed: the clock that starts the settle, verify and refresh jobs.

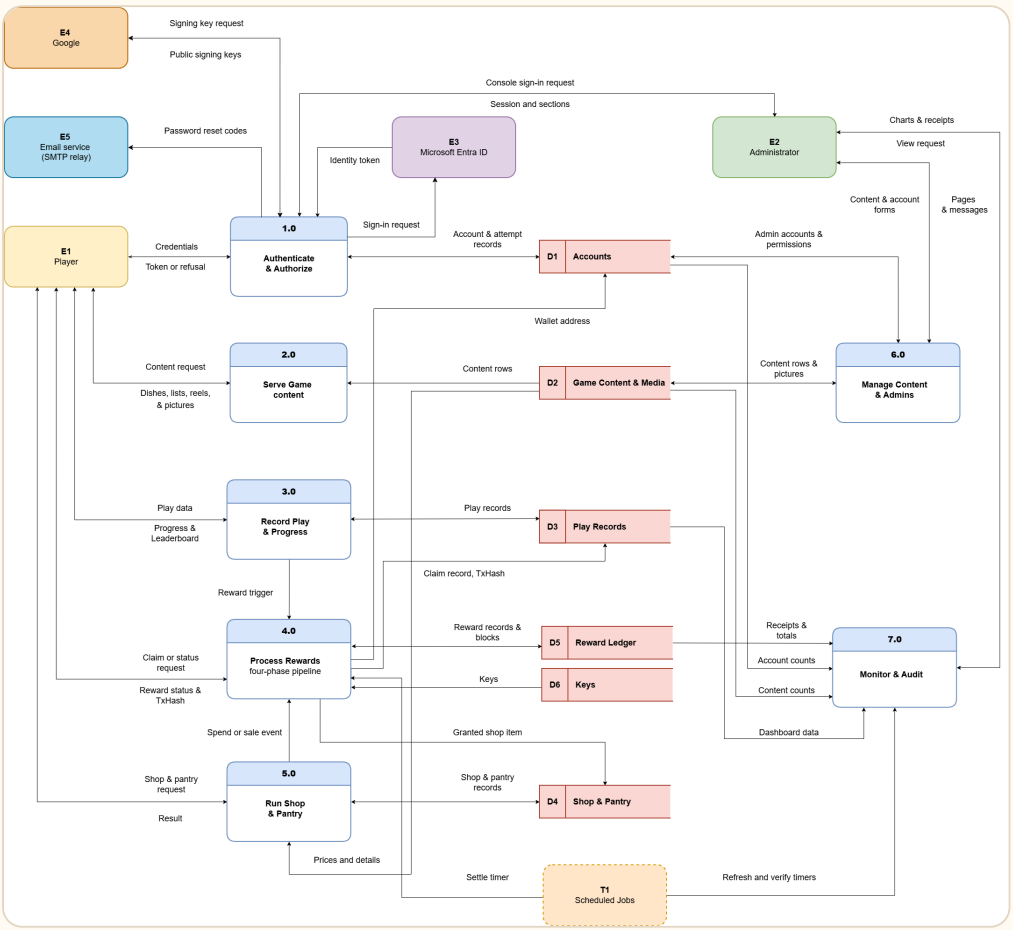
RULES TO CHECK

- Every process has at least one input **and** one output (no "black holes" or "miracles").
- Data never moves store to store, entity to store, or entity to entity without a process in between.
- **Balancing:** the flows into and out of a Level 1 process must match its Level 2 diagram.
- Labels are nouns (what the data is), never verbs (that is the process's job).
- Other notation you may see: Yourdon/DeMarco draws processes as circles and stores as two parallel lines. Same meaning.

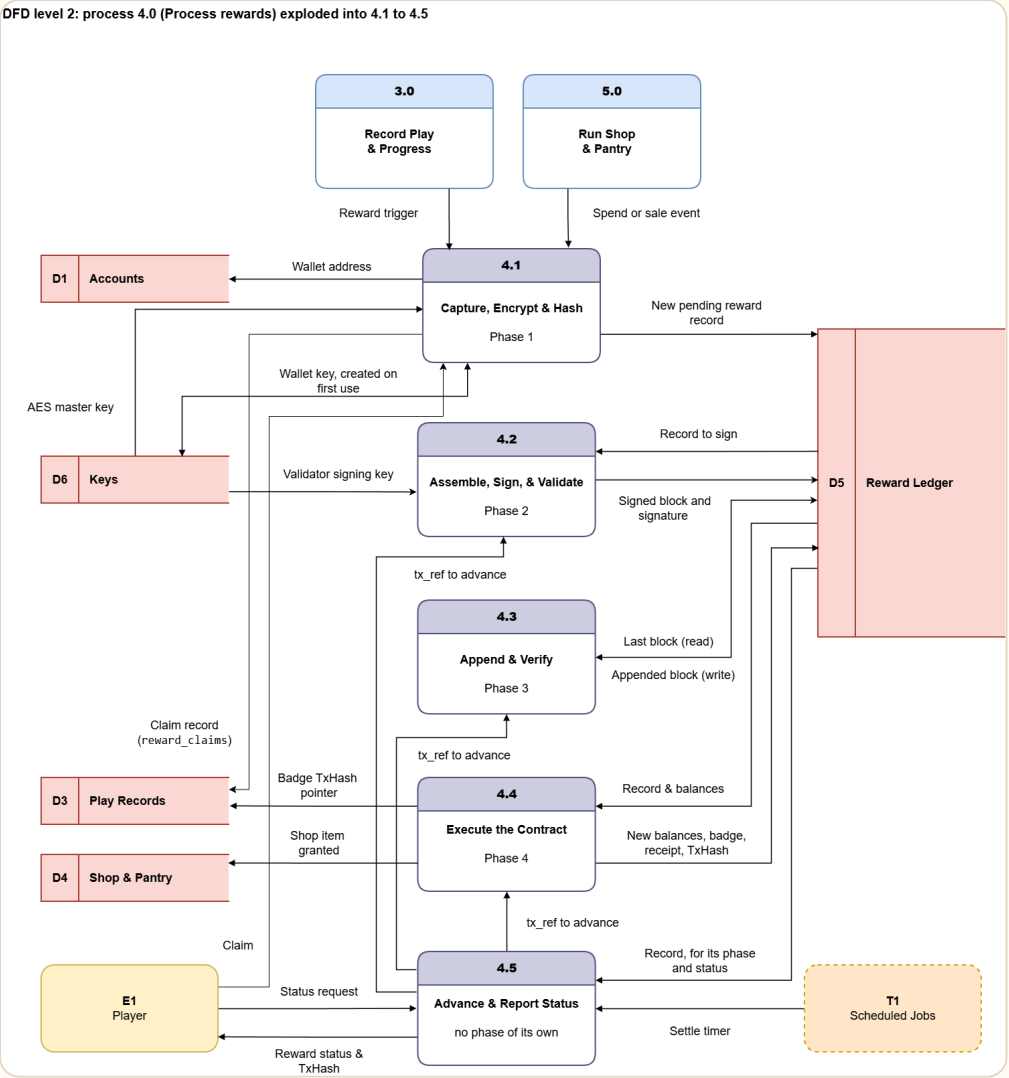
OUR DIAGRAMS



Level 0: the whole system as one process (0), with the five outside parties and the timer.



Level 1: seven processes (1.0 to 7.0) and six data stores (D1 to D6).



Level 2: process 4.0 Process Rewards exploded into 4.1 to 4.5.

HOW TO READ OURS

- **Level 0:** the Player (E1) and the Administrator (E2) send requests and get results back. Microsoft Entra ID (E3) signs admins in; Google (E4) gives the public keys that prove a player's Google token is real; the Email service (E5) sends reset codes; the timer (T1) starts the scheduled jobs.
- **Level 1:** 1.0 Authenticate, 2.0 Serve game content, 3.0 Record play, 4.0 Process rewards, 5.0 Run shop and pantry, 6.0 Manage content and admins, 7.0 Monitor and audit. The stores are D1 Accounts, D2 Game content and media, D3 Play records, D4 Shop and pantry, D5 Reward ledger, D6 Keys.
- **Level 2:** the four reward phases (4.1 to 4.4) and 4.5, which moves a record to its next phase each time the app asks for its status (or when the 15-minute settle job runs).

IN PLAIN WORDS

Follow the paperwork, not the people: who hands in which form, which desk works on it, and which filing cabinet it ends up in.

PANEL MIGHT ASK

Why is the Scheduler drawn at all? Some processes start on a clock, not on a user: settling rewards every 15 minutes, verifying the chain every hour, refreshing the receipt copy every minute. T1 shows where those start.

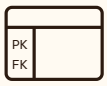
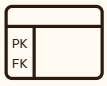
Where are the reports? The reports feature is turned off on the adviser's advice, so its table and flows are left out.

6.2 Entity relationship diagram (ERD) `crow's foot · 35 tables`

WHAT IT IS

An ERD shows the **database design**: each table (entity), its columns (attributes), its keys, and how tables relate. Ours uses **crow's foot** notation and is the physical model, so it uses the real table and column names from Azure SQL.

SYMBOLS

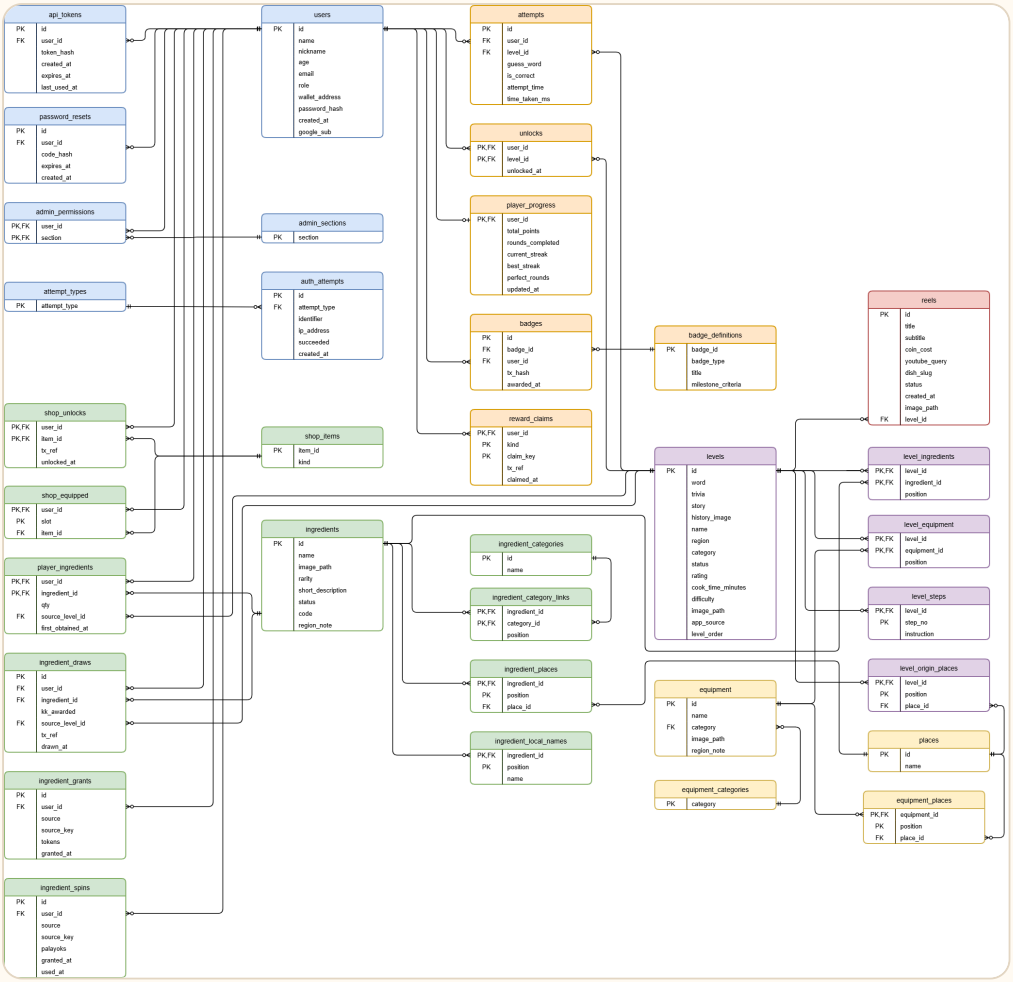
SYMBOL	NAME	WHAT IT MEANS
	**Entity (table)**	One table. The header is its name; each row is a column.
	**PK / FK**	**PK** (primary key) uniquely identifies a row. **FK** (foreign key) points to another table's PK. "PK,FK" means the column is both, as in a junction table.
	**Exactly one**	Two bars: one and only one (mandatory).
	**Zero or one**	Circle and bar: optional, at most one.
	**One or many**	Bar and crow's foot: at least one.
	**Zero or many**	Circle and crow's foot: any number, including none.

HOW TO READ A LINE

Read each end from the table on the **other** side. `users` ||——o< `attempts` says: one attempt belongs to exactly one user, and one user has zero or many attempts.

A **many-to-many** relationship cannot be stored directly, so it goes through a **junction table** whose key is both FKs: `level_ingredients` (level_id, ingredient_id) links dishes to ingredients.

OUR DIAGRAM



All 35 tables, colour-coded by area. Checked against the Azure SQL schema on 15 Oct 2026: every table and column matches.

- **Blue, accounts (7):** `users`, `api_tokens`, `password_resets`, `auth_attempts` and `attempt_types` (the lockout), `admin_permissions` and `admin_sections`.
- **Orange, play and rewards (6):** `attempts`, `unlocks`, `player_progress`, `badges` and `badge_definitions`, `reward_claims` (the "only once" rule for rewards).
- **Green, shop, pantry and ingredients (12):** `shop_items`, `shop_unlocks`, `shop_equipped`, the four pantry tables, `ingredients` and its categories, places and local names.
- **Purple, dishes (5):** `levels` and its child tables for ingredients, equipment, steps and places of origin.
- **Yellow and red:** the equipment catalog and `places`; `reels` (linked to a dish by `level_id`).

IN PLAIN WORDS

A set of spreadsheets that point at each other by ID, so each fact is written once: change a dish's name in one row and every player sees it.

PANEL MIGHT ASK

Is it normalized? Yes, to third normal form. On 12 to 13 Oct, lists stored as comma-separated text (a dish's ingredients, an admin's sections) were moved into child tables such as `level_ingredients` and `admin_permissions`.

Where is the blockchain? Not in SQL. The ledger lives in files on the API VM (`blocks.jsonl` and the receipts). The database keeps only pointers such as `badges.tx_hash` and `users.wallet_address`.

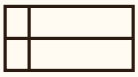
Why is `reports` missing? The table still exists, but the feature is hidden in the app and console on the adviser's advice, so it is left out of the ERD.

6.3 Process flow `swimlane flowchart · PF1 to PF4`

WHAT IT IS

A process flow is a **flowchart with swimlanes** (a cross-functional flowchart). It follows **one scenario** from start to end and shows which part of the system does each step. Ours have lanes for the Player or Administrator, the App or Console, the API, the Database and the Ledger.

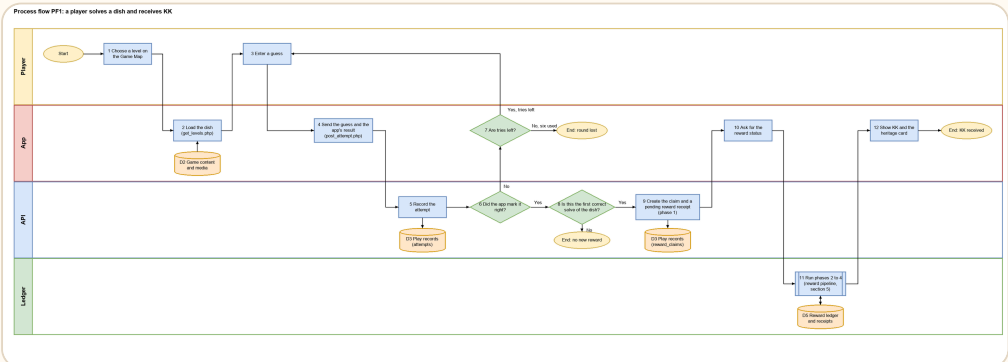
SYMBOLS

SYMBOL	NAME	WHAT IT MEANS
	**Terminator**	Start or End of the scenario. Every path ends in one: "End: KK received", "End: round lost".
	**Process step**	One action, numbered in order: "5 Record the attempt".
	**Decision**	A yes/no question. Each exit is labelled with its answer.
	**Data store**	Where the step reads or writes data (D3 Play records). Uses the same IDs as the DFD.
	**Predefined process**	A step explained in another diagram: "11 Run phases 2 to 4" is the reward pipeline (6.4).
	**Swimlane**	One row per actor or part of the system. When the arrow crosses a lane, the work is handed over.
	**Flow line**	The next step.

HOW TO READ IT

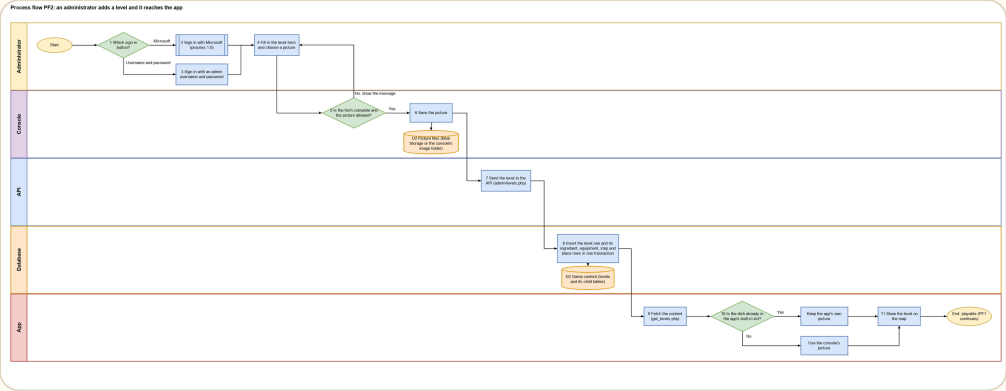
- Start at the Start oval and follow the numbers.
- Every time the line changes lane, say who hands what to whom ("the app sends the guess to the API").
- At each diamond, follow one answer to its End, then come back for the other.

OUR DIAGRAMS



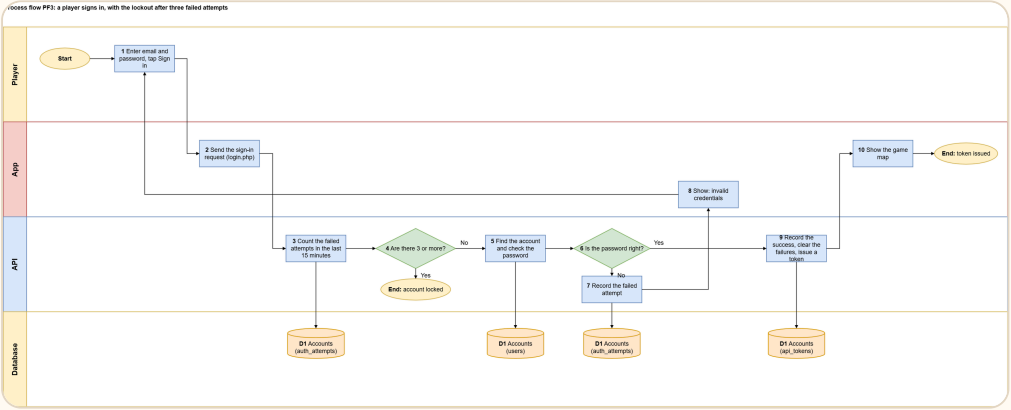
PF1: a player solves a dish and receives KK.

The app checks each guess and sends it with its result to `post_attempt.php`; the API records every attempt. On a correct first solve the API inserts the claim row and a pending receipt (Phase 1), and the remaining phases run as the app asks for the reward status.



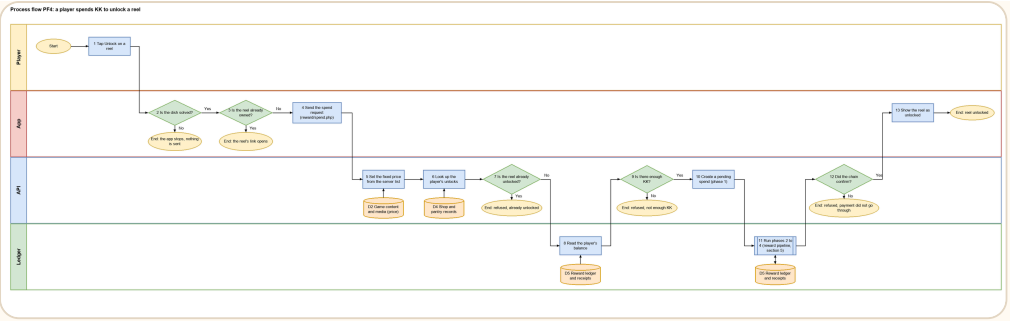
PF2: an administrator adds a level and it reaches the app.

Sign in (Microsoft or the emergency account), fill in the form, the console checks it and saves the picture, the API writes the level and its ingredient, equipment, step and place rows in one transaction, and the app shows it on the Game Map. The API checks that the account is an administrator; the Levels permission is enforced by the console hiding the tab.



PF3: a player signs in, with the lockout after three failed attempts.

The API first counts the account's failed attempts in the last 15 minutes. Three or more means locked (HTTP 429: "reset your password, or try again in 15 minutes"). Otherwise it checks the password; a failure is recorded, a success clears the failures and issues a token. The lock follows the account typed in, not the phone.



PF4: a player spends KK to unlock a reel.

The app stops early if the dish is unsolved or the reel is already owned. The API sets the price itself (the app cannot choose it), refuses a reel already unlocked, checks the balance, and runs all four reward phases in the same request, so the reel is only granted after the payment is confirmed.

IN PLAIN WORDS

A relay race drawn on paper: each lane is a runner, and the arrow shows exactly when the baton is passed.

PANEL MIGHT ASK

Why does PF4 finish in one request but PF1 does not? A spend must be paid before the reel is given, so the API runs all four phases before answering. A win only adds KK, so the app gets a pending receipt at once and the remaining phases run as it asks for the status.

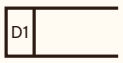
6.4 Data pipeline `transaction lifecycle · batch ETL`

WHAT IT IS

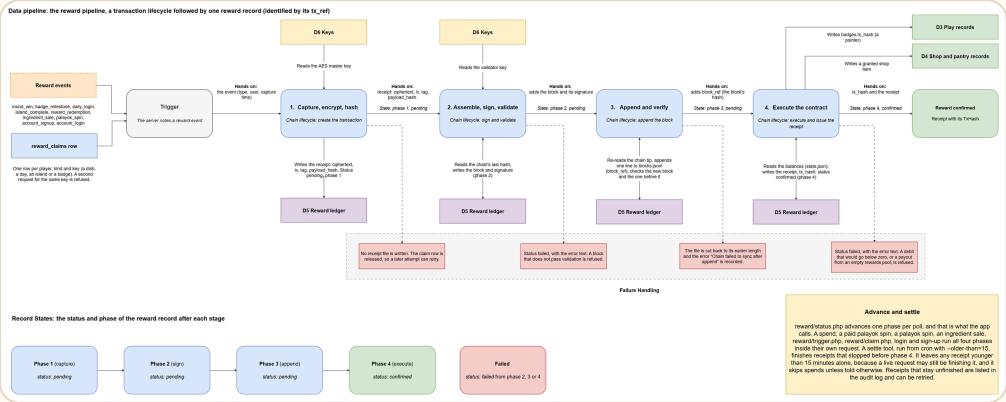
A data pipeline is a chain of **stages** that data passes through, where each stage takes the output of the one before. Two common kinds:

- **Transaction lifecycle** (event processing): one record moves through fixed phases and states, like our reward pipeline.
- **Batch ETL**: scheduled jobs **E**xtract data from a source, **T**ransform it (sort, count, check), **L**oad the result somewhere fast to read, and then **serve** it, like our reporting pipeline.

SYMBOLS

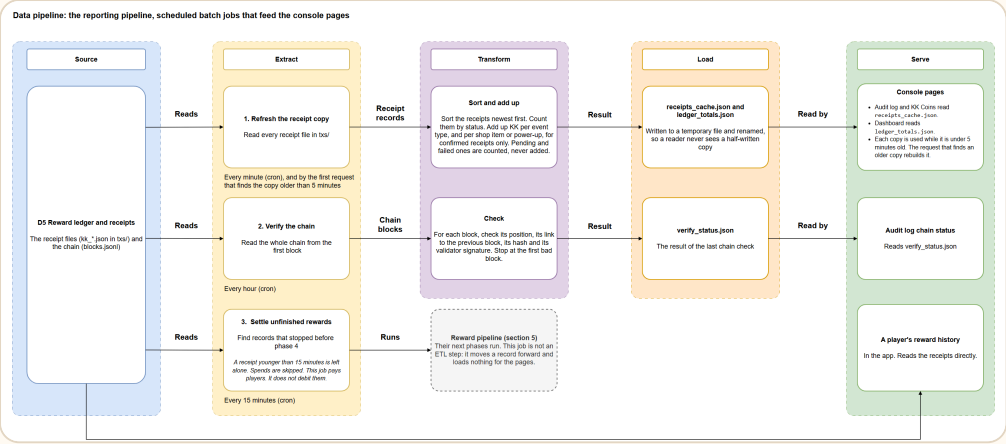
SYMBOL	NAME	WHAT IT MEANS
	**Stage**	One step of the pipeline, in order from left to right.
	**Data store**	Where a stage reads or writes (D5 Reward ledger, D6 Keys).
	Hand-off	The record or data passed to the next stage ("Hands on: tx_hash and the receipt").
	Failure path	Dashed: what happens if the stage fails.
	**Trigger**	What starts it: a request, or a schedule (cron: every minute, every hour, every 15 minutes).

OUR DIAGRAMS



Reward pipeline: one reward record, followed by its `tx_ref`, through the four phases.

- **Phase 1, capture, encrypt, hash:** the event is sealed with AES-256-GCM and fingerprinted with SHA-256. State: pending.
- **Phase 2, assemble, sign, validate:** a block is built and signed with the validator's ECDSA key, and the PoA rule checks it.
- **Phase 3, append and verify:** the block is added to `blocks.jsonl`, linked to the one before.
- **Phase 4, execute the contract:** KKCoin and KKBadge rules update the balance, write the TxHash receipt, and grant a badge or shop item. State: confirmed.
- **Failures:** each phase has its own failure path (dashed); a failed record is kept and listed in the audit log. Phase 4 refuses a debit that would go below zero, or a payout when the rewards pool is empty.



Reporting pipeline: the scheduled jobs that feed the console.

- **Every minute:** read every receipt, sort and add up, and write `receipts_cache.json` and `ledger_totals.json` for the Audit log, KK Coins and Dashboard pages.
- **Every hour:** verify the whole chain (position, link, hash and signature of each block) and write `verify_status.json`.
- **Every 15 minutes:** settle rewards that stopped before Phase 4 (older than 15 minutes; spends are skipped).

IN PLAIN WORDS

The reward pipeline is a receipt going through four stamps. The reporting pipeline is the night clerk who adds up all the receipts so the manager's screen opens fast.

PANEL MIGHT ASK

Why a saved copy instead of reading the receipts live? During the stress test, opening every receipt file on each page load took 6 to 10 seconds. The copy rebuilt every minute brought it under 0.75 seconds.

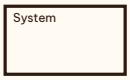
Can the settle job pay twice? No. It skips records younger than 15 minutes (a live request may still be finishing them), and the claim row allows each reward only once.

6.5 Use case diagram UML

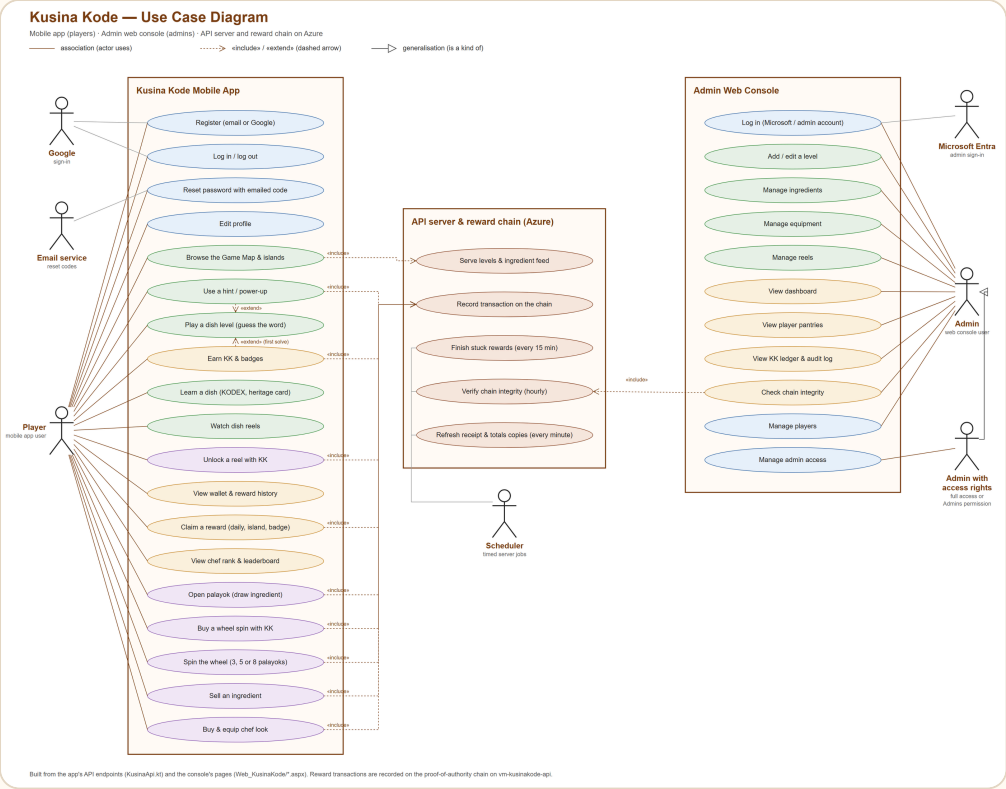
WHAT IT IS

A use case diagram shows **who** uses the system (actors) and **what goals** they can reach with it (use cases). It shows scope, not steps: the order of steps is in the activity diagrams.

SYMBOLS

SYMBOL	NAME	WHAT IT MEANS
	**Actor**	A role outside the system, not a specific person: Player, Admin. Outside systems can be actors too (Google, Microsoft Entra, the Scheduler).
	**Use case**	A goal written as a verb phrase: "Play a dish level".
	**System boundary**	The box around what the system does. Actors stay outside it.
	Association	Solid line: the actor takes part in that use case.
	«include» / «extend»	Dashed arrow. **«include»**: the base use case **always** does the other one (arrow points to the included one). **«extend»**: an **optional or conditional** add-on (arrow points to the base).
	**Generalisation**	Hollow triangle: "is a kind of". The child actor can do everything the parent can, and more.

OUR DIAGRAM



Three boundaries: the mobile app, the API server and reward chain, and the admin web console.

- **Player:** register and sign in, play a dish, use a hint, learn a dish, watch reels, claim rewards, spin the wheel, open palayoks, sell ingredients, buy chef looks.
- **«extend» examples:** *Use a hint / power-up* extends *Play a dish level* (optional); *Earn KK & badges* extends it too, because KK is paid only on the **first** solve.
- **«include» examples:** every KK action includes *Record transaction on the chain*; browsing the Game Map includes *Serve levels*.
- **Admin** manages levels, ingredients, equipment and reels and views the dashboard and ledger. **Admin with access rights** (generalisation) can also manage admin access.
- The **Scheduler** runs the timed jobs: finish stuck rewards (15 min), verify the chain (hourly), refresh the receipt copy (every minute).

IN PLAIN WORDS

A menu of what each kind of user can order from the system, without the recipe for how it is cooked.

PANEL MIGHT ASK

Why is Earn KK «extend» and not «include»? «Include» would mean every round pays KK. It doesn't: only the first correct solve of each dish does.

6.6 System architecture 3 tiers

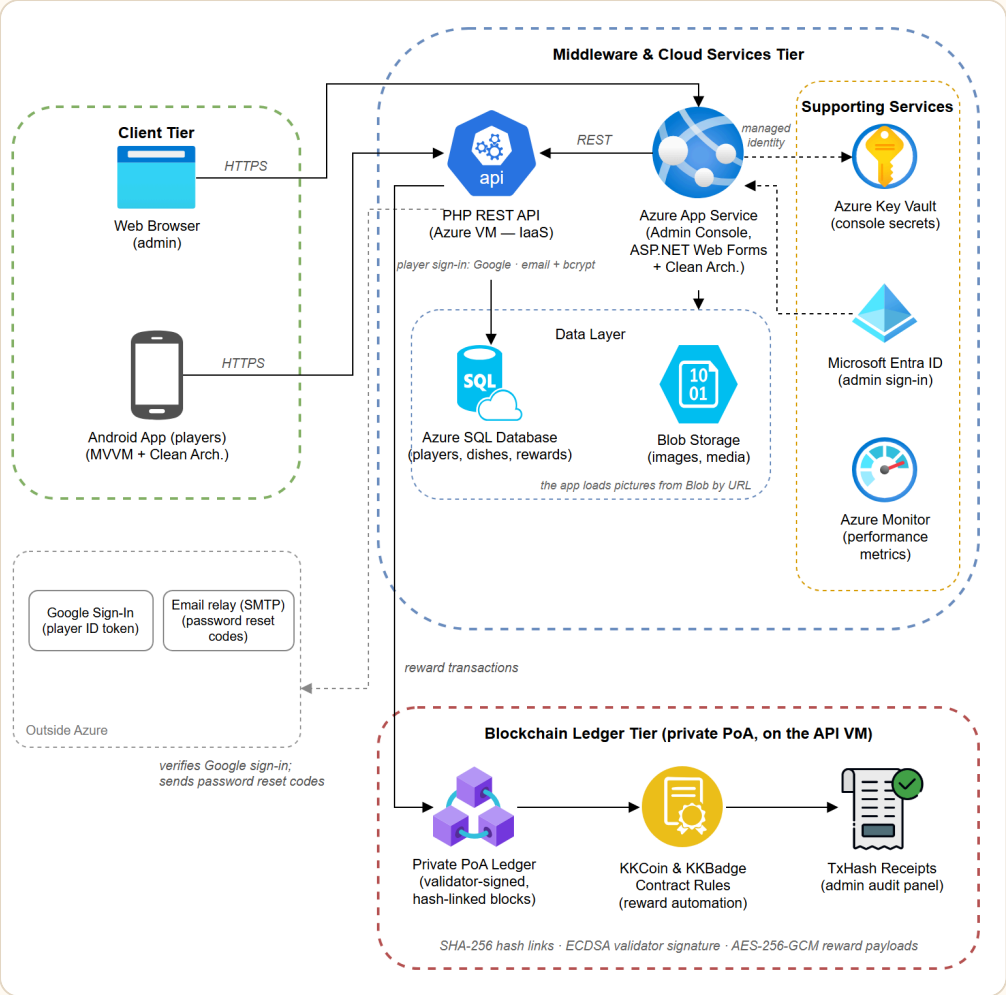
WHAT IT IS

A system architecture diagram shows the **main parts** of the system and how they **connect**, usually grouped into tiers (layers). Each tier has one job and talks to the tier next to it.

SYMBOLS

SYMBOL	NAME	WHAT IT MEANS
	**Tier / boundary**	A dashed group: one layer (Client, Middleware and Cloud Services, Blockchain Ledger) or "Outside Azure".
	**Component**	One part of the system, usually shown with its product icon (the API, Azure SQL, Key Vault).
	**Connector**	Who calls whom, labelled with the protocol or the data (HTTPS, REST, reward transactions). The arrow points from the caller.
	**Dashed connector**	A supporting link, such as a managed identity or a call to an outside service.

OUR DIAGRAM



Client tier, middleware and cloud services tier, and the blockchain ledger tier.

- **Client tier:** the admin's web browser and the Android app (MVVM + Clean Architecture), both over HTTPS.
- **Middleware and cloud services:** the PHP REST API on an Azure VM and the Admin Console on App Service (ASP.NET Web Forms + Clean Architecture); the data layer (Azure SQL, Blob Storage); supporting services (Key Vault through the console's managed identity, Entra ID for admin sign-in, Azure Monitor).
- **Outside Azure:** Google Sign-In (the API checks the player's ID token) and the email relay (password reset codes).
- **Blockchain ledger tier:** only the API sends reward transactions into it: the private PoA ledger, then the KKCoin and KKBadge contract rules, then the TxHash receipts.

IN PLAIN WORDS

The front counter (apps) takes orders, the kitchen (API and database) prepares them, and the cashier's sealed book (ledger) records every payment. Customers never write in the book themselves.

PANEL MIGHT ASK

Why separate tiers? Each tier can change without breaking the others. The database was normalized on 12 to 13 Oct without changing the app, because the API kept the same replies.

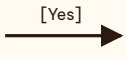
6.7 Activity diagram

UML · player and administrator

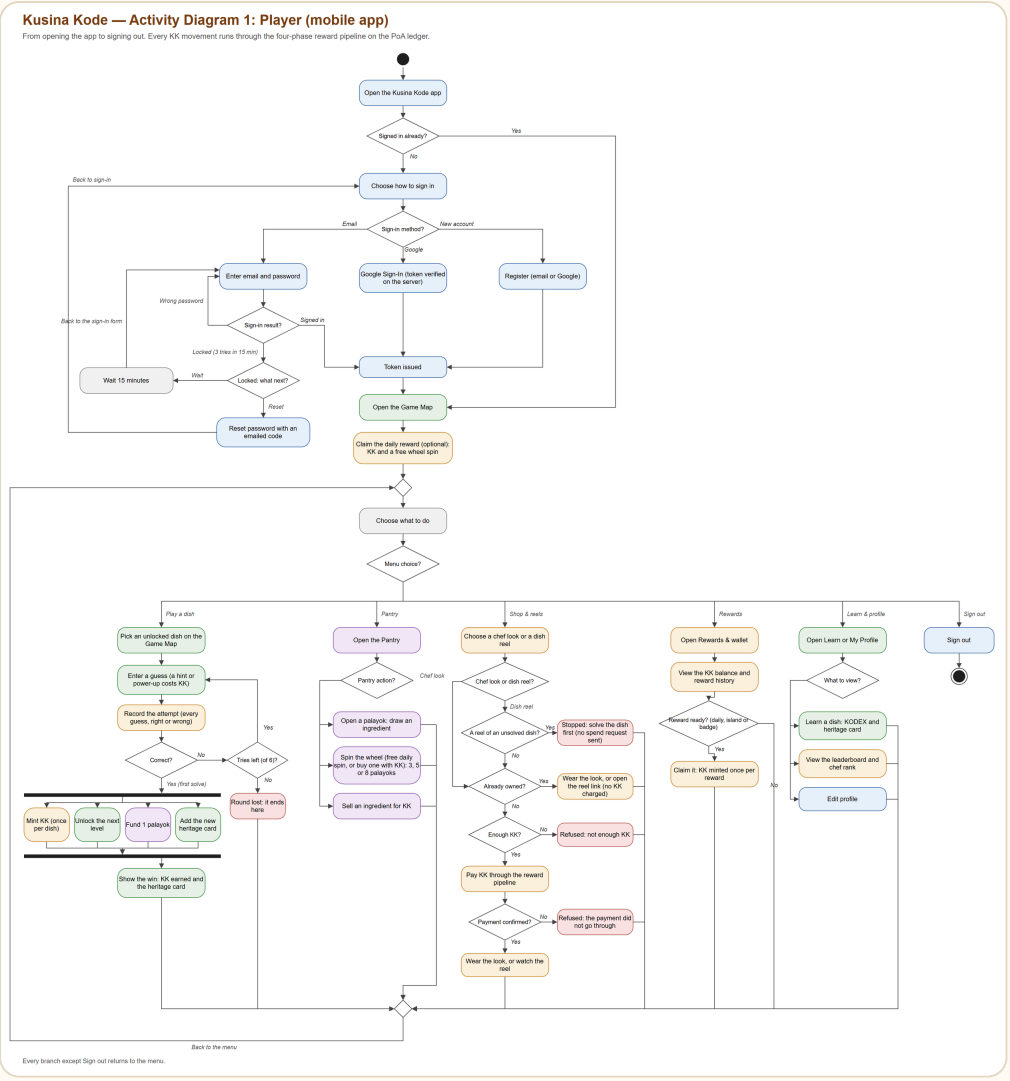
WHAT IT IS

An activity diagram (UML) shows **every path** a user can take through a feature: the actions, the decisions, the loops, and the steps that happen together. Think of it as a flowchart with extra UML rules for parallel work.

SYMBOLS

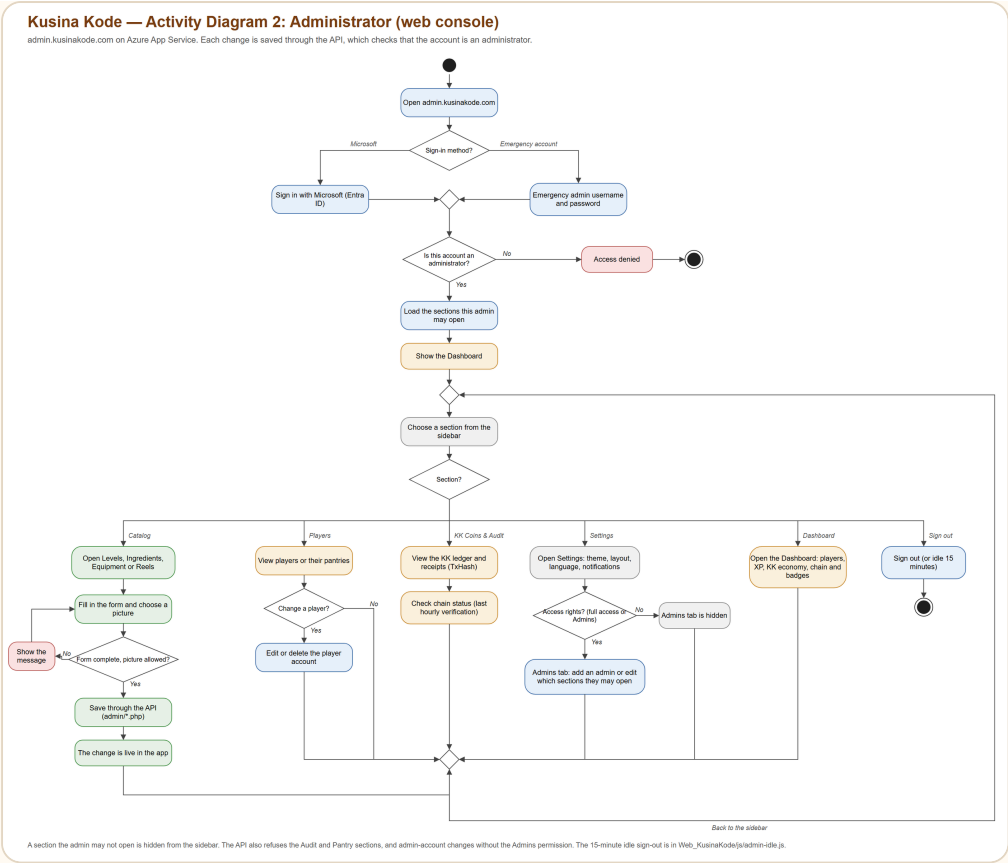
SYMBOL	NAME	WHAT IT MEANS
	**Initial node**	Where the activity starts. Only one.
	**Activity final**	Where it ends (bullseye).
	**Action**	One step, in a rounded box: "Enter a guess".
	**Decision**	One way in, several ways out; each exit has a guard.
	**Merge**	Several ways in, one way out: the branches meet again (the small diamond before "Choose what to do").
	**Fork / join**	Thick bar. A **fork** starts steps that all happen; a **join** waits until all of them are done (mint KK, unlock the next level, fund a palayok, add the heritage card).
	**Guard**	The condition on an exit, such as [Yes], [No] or [Locked]. Ours are written in italics beside the line.

OUR DIAGRAMS



Player (mobile app): from opening the app to signing out.

- **Sign-in:** email, Google or a new account. A wrong password loops back; after 3 tries in 15 minutes the account is locked, and the player either **waits 15 minutes** or **resets the password** with an emailed code.
- **Play a dish:** every guess is recorded, right or wrong. A wrong guess asks "Tries left (of 6)?"; the first correct solve forks into minting KK, unlocking the next level, funding one palayok and adding the heritage card.
- **Shop and reels:** the first question is *chef look or dish reel?* Only a reel can belong to an unsolved dish, which stops the purchase (no spend is sent). Then both meet at *Already owned?*: an owned look is worn and an owned reel opens its link, with no KK charged. Otherwise the KK check and the payment follow.
- Every branch except Sign out returns to the menu.



Administrator (web console): from signing in to signing out.

- Sign in with Microsoft (Entra ID) or the emergency account; a non-admin account is refused.
- The sidebar shows only the sections this admin may open: the catalogs, players, KK coins and audit, settings, and the dashboard. The Admins tab appears only with the Admins permission.
- Sign out, or the console signs out after 15 minutes idle.

IN PLAIN WORDS

A "choose your own adventure" map of the app: every choice the user can make, and where each one leads.

PANEL MIGHT ASK

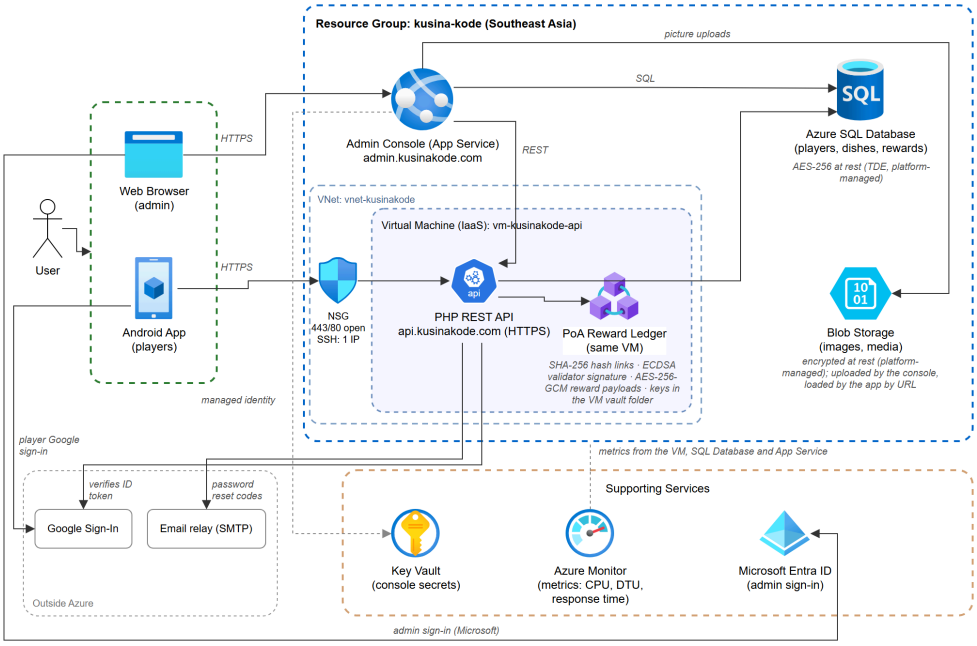
Why a fork bar after a correct guess? The four rewards of a first solve are independent and all must happen; a fork and join shows "all of these" better than four boxes in a row.

6.8 Other diagrams `cloud · cybersecurity · conceptual framework`

CLOUD ARCHITECTURE

Shows **where each part runs**, using the provider's official icons and its containers: the resource group, the virtual network (VNet), the VM, the firewall rules (NSG). The service model matters: an **IaaS** VM for the API and ledger, **PaaS** for the console, database and storage.

SYMBOL	NAME	WHAT IT MEANS
	**Container**	Dashed groups nest: resource group, then VNet, then VM. Anything inside is managed or reached through its container.
	**Connection**	A network call, labelled with its protocol or purpose.

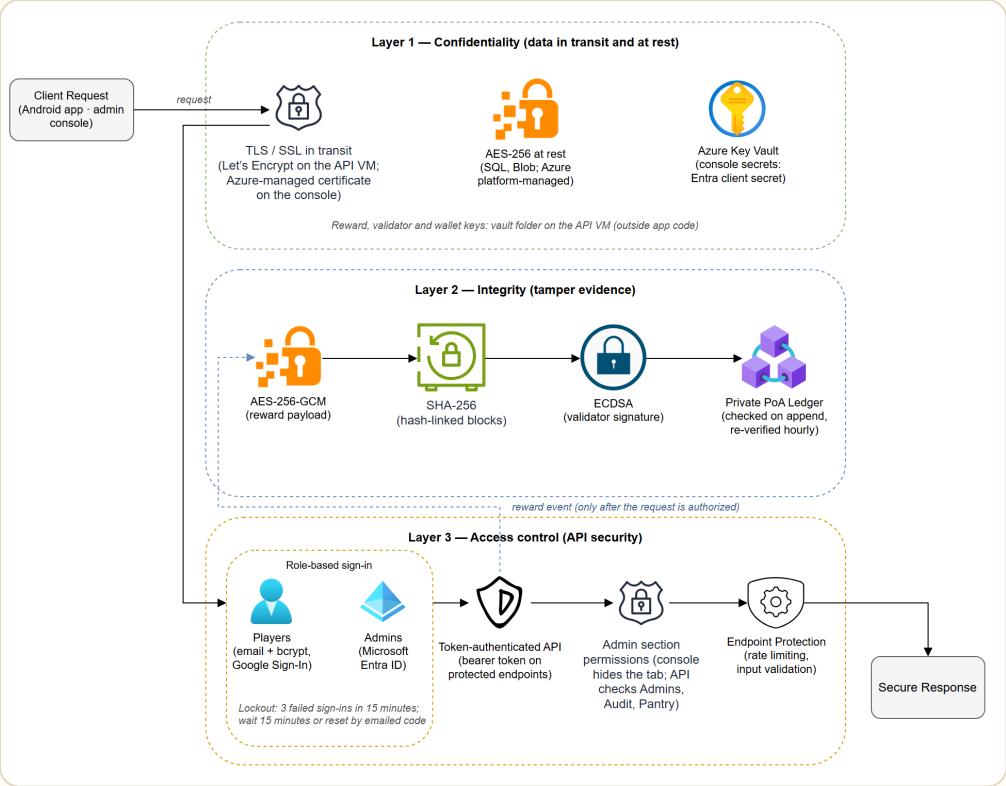


Every Azure resource in the `kusina-kode` resource group (Southeast Asia), plus the services outside Azure.

- Admins reach the console (App Service, `admin.kusinakode.com`) and sign in through Entra ID; the console reads its secret from Key Vault through its managed identity.
- Players reach the API through the NSG (443/80 open, SSH from one IP) on the VM, where the PoA ledger also runs.
- The console uploads pictures to Blob Storage; the app loads them by URL. Azure Monitor collects metrics from the VM, SQL Database and App Service.

CYBERSECURITY ARCHITECTURE

Shows the security controls grouped by the **CIA triad**: Confidentiality (data in transit and at rest), Integrity (tamper evidence), and Access control (who may do what).



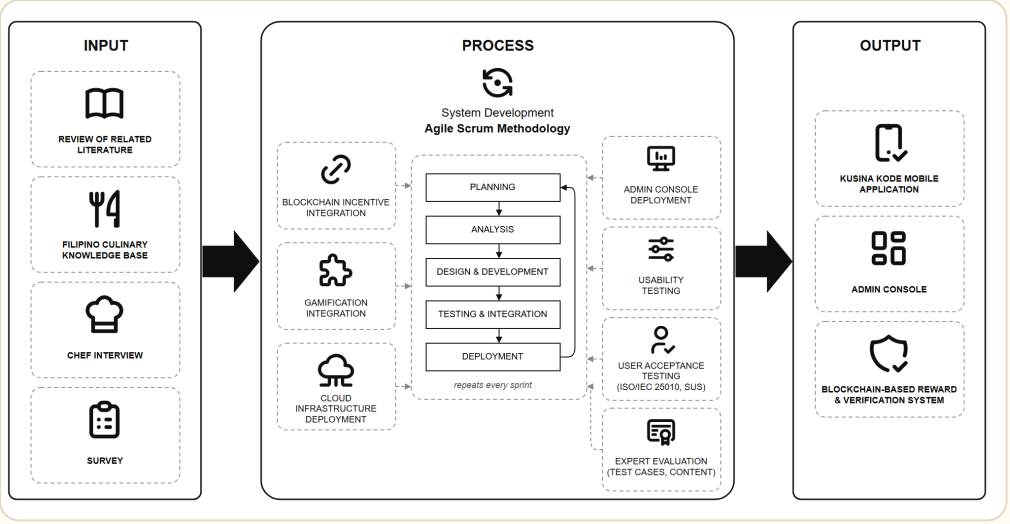
Three layers that every request passes through.

- **Layer 1:** TLS (Let's Encrypt on the API, an Azure-managed certificate on the console), AES-256 at rest, Key Vault for the console's secrets; the reward, validator and wallet keys sit in a vault folder on the VM.
- **Layer 2:** AES-256-GCM payloads, SHA-256 hash links and ECDSA signatures, checked when appended and re-verified hourly.
- **Layer 3:** role-based sign-in with the 3-tries lockout, a token on protected endpoints, section permissions (the console hides tabs; the API also checks Admins, Audit and Pantry), and rate limiting with input validation.

CONCEPTUAL FRAMEWORK

The research model, drawn as **Input** → **Process** → **Output** (IPO): what the study started from, what the team did, and what it produced.

SYMBOL	NAME	WHAT IT MEANS
	**IPO boxes**	Read left to right. The arrows mean "leads to", not data or network traffic.



Input, process (Agile Scrum, repeated every sprint) and output.

- **Input:** the review of related literature, the Filipino culinary knowledge base, the chef interview, and the survey.
- **Process:** Scrum (planning, analysis, design and development, testing and integration, deployment) repeated every sprint, with blockchain, gamification and cloud work on one side and the admin console, usability testing, UAT (ISO/IEC 25010 and SUS) and expert evaluation on the other.
- **Output:** the mobile app, the admin console, and the blockchain-based reward and verification system.

PANEL MIGHT ASK

Cloud vs system architecture? The system architecture shows the logical parts and tiers; the cloud architecture shows the actual Azure resources they run on.

Why is it called a "conceptual" framework? It shows how the study's ideas connect, not how the software is built.

Diagrams: DFD, ERD, process flows and data pipelines by Kyla (14 to 15 Oct 2026); use case, system architecture, activity diagrams, cloud and cybersecurity architecture and the conceptual framework by Mau (draw.io, 15 Oct 2026). Checked against the API, console and app code and the Azure SQL schema on 15 Oct 2026.